

02a — Single-Label Transitions

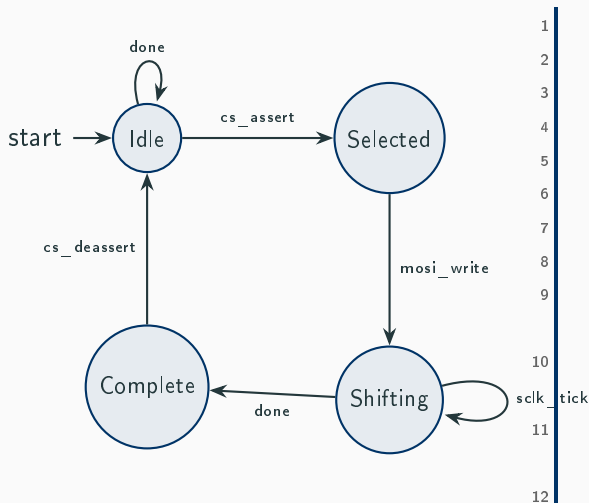
Controllable Labels and Controller Synthesis

Mariano Cerrutti

Mununu Tutorial

SPI Master Automaton

A Serial Peripheral Interface master with four states:



```
1 states {  
2     state Idle initial;  
3     state Selected;  
4     state Shifting;  
5     state Complete;  
6 }  
7  
8 transitions {  
9     // Assert chip-select to  
10    begin transaction  
11    transition Idle -> Selected  
12    on label cs_assert;  
    // Idle self-loop when no  
    transaction pending  
    transition Idle -> Idle on 1/5
```

Controllable Labels

```
1      controllable {  
2          label cs_assert;  
3          label cs_deassert;  
4          label sclk_tick;  
5          label mosi_write;  
6          label done;  
7      }
```

- A `controllable` block declares labels the **controller can choose** to fire.
- Labels *not* listed here are `uncontrollable` — they come from the environment.
- This distinction is the foundation of **controller synthesis**: find a strategy that only uses controllable actions to guarantee a property.

Labeled Modalities

```
1 // Labeled modality: from Idle, cs_assert leads to Selected
2 formula cs_leads_to_selected {
3     over SpiMaster;
4     body = (! Idle) || < labels = { cs_assert } > Selected;
5 }
```

The diamond modality can be **filtered by label set**:

$\langle \text{labels} = \{\text{cs_assert}\} \rangle \text{Selected}$

“There *exists* a transition labeled `cs_assert` that leads to a state satisfying *Selected*.”

Compare with the unrestricted diamond $\Diamond \varphi$ which considers *all* outgoing transitions.

Controller Synthesis

```
1   controllers {  
2       controller spi_safety {  
3           source SpiMaster;  
4           satisfying safety_invariant;  
5       }  
6   }
```

- `controller` names a synthesis task.
- `source` selects the automaton.
- `satisfying` references the property to enforce.

Run synthesis:

```
$ mununu context synth 02a_single_label.ctxdsl \  
    --controller spi_safety  
Controller spi_safety: REALIZABLE
```

New Syntax Summary

Keyword	Purpose
<code>controllable</code>	Declares labels the controller may choose
<code>controllers</code>	Block of controller synthesis tasks
<code>controller</code>	One named synthesis task
<code>source</code>	The automaton to synthesize a controller for
<code>satisfying</code>	The property the controller must enforce

Key idea: Controllable vs. uncontrollable labels let us ask “can we *choose* actions to guarantee a property, regardless of what the environment does?”